HATHOR-CORE · MASTER-BOOK

Consensus — Which History Wins

How the node decides the canonical ledger when vertices conflict or chains compete — accumulated weight, voiding, reorgs, and the proof-of-authority variant.

SUBJECT	hathor-core · Part II · the node, end to end
CHAPTER	32 · Part II · The Node
BRANCH	study/hathor-core-studies
EDITION	2026 · v1

Consensus · ConsensusAlgorithm · Accumulated weight · Score · voided_by · Conflict resolution · Reorg · Block vs tx consensus · Proof-of-Authority

Table of Contents

Chapter 32 — Consensus: Which History Wins

What you'll learn in this chapter

- The difference between **verification** (Ch 31, "is this vertex valid on its own?") and **consensus** ("given everything else I know, which history is the real one?").
- How Hathor records a "this is not part of the real ledger" decision without deleting anything, using the `voided_by` set on a vertex's metadata.
- How two conflicting transactions are resolved: the one with the **higher accumulated weight** wins, the loser is **voided**, and that voiding **propagates** to everything downstream of the loser.
- How competing chains of blocks are resolved by **score**, and what a **reorg**[1] (reorganization) is — when the node abandons one chain of blocks for a heavier one.
- Why **block voiding behaves differently from transaction voiding** — an asymmetry that trips up most readers.
- How Hathor's alternative consensus engine, **Proof-of-Authority**[2] (PoA), replaces mining with a fixed set of authorized block signers — used on private networks.

This is one of the load-bearing chapters of the book. Chapter 10 gave you the vocabulary — conflict, voiding, reorg, finality — at the conceptual level. This chapter pays that vocabulary off against the real code in `hathor/consensus/`. By the end you should be able to point at the exact method that decides a conflict, trace by hand what happens when a heavier transaction arrives, and explain why the algorithm marks losers instead of erasing them.

32.1 Localization

The consensus package is small in file count and dense in logic. It sits in the domain model group of the codebase, right next to verification, because together they answer the two questions a node asks of every incoming vertex.

```
hathor-core/
└── hathor/
    ├── verification/            ← "is this vertex valid in isolation?" (Ch 31)
    │
    ├── consensus/               ◀ YOU ARE HERE  — "which history is canonical?"
    │   ├── __init__.py          ← exports ConsensusAlgorithm
    │   ├── consensus.py         ← ConsensusAlgorithm: the orchestrator
    │   ├── context.py           ← ConsensusAlgorithmContext + ReorgInfo
    │   ├── block_consensus.py   ← BlockConsensusAlgorithm: chain selection, score, reorg
    │   ├── transaction_consensus.py  ← TransactionConsensusAlgorithm: conflict resolution
    │   ├── consensus_settings.py     ← PowSettings / PoaSettings (which engine?)
    │   └── poa/                 ← the Proof-of-Authority variant
    │       ├── poa.py           ← signer set, weight rule, signature verification
    │       ├── poa_signer.py    ← PoaSigner: signs PoA blocks
    │       └── poa_block_producer.py  ← the PoA equivalent of a miner
    │
    └── vertex_handler/          ← the pipeline that runs verification THEN consensus (Ch 33)
```

Recap — verification vs. consensus (full treatment in Ch. 10 & Ch. 31). Verification asks a local question about one vertex: are its signatures valid, is its proof-of-work sufficient, does it spend coins that exist, is it well-formed? A vertex either passes or it is rejected outright — and a rejected vertex never enters the ledger. Consensus asks a global question: given a vertex that is already valid, does accepting it change which version of history the node considers real? A vertex can be perfectly valid and still end up voided by consensus — for example, because a competing valid transaction spends the same coin and happens to carry more weight. Verification produces a yes/no; consensus produces a ranking. → full treatment of the conceptual split in Ch. 10 §10.1.

Context. Consensus is where "data arrived and checked out" becomes "the ledger now says X." It is invoked once per accepted vertex, immediately after verification, by the vertex handler (Ch 33). It never validates — it assumes the vertex is already valid — and it writes its decisions into the metadata the storage layer keeps for each vertex (Ch 25): `voided_by`, `accumulated_weight`, `score`, `first_block`. Every downstream consumer — the indexes (Ch 28), the wallet, the APIs — reads those fields to know what is "really" in the ledger.

32.2 What it does and why it exists

A full node receives vertices from many peers, in unpredictable order, with no global clock. Two honest users on opposite sides of the world can each spend the same coin at nearly the same instant; both transactions are individually valid; both reach the node. Verification cannot reject either — each one, looked at alone, is legitimate. Something

has to decide which of the two the ledger will honor, and that decision must be **the same on every node in the network**, or the network forks into disagreeing copies of reality.

That "something" is the consensus algorithm. Its job, stated precisely, is to assign every vertex one of two states:

- **executed** — this vertex is part of the canonical ledger; its outputs are spendable, its effects count.
- **voided** — this vertex is not part of the canonical ledger right now; its effects are suppressed.

Hathor encodes this with a single piece of metadata, `voided_by`, a set of vertex hashes (`transaction_metadata.py:45`). The rule is stated at the top of the orchestrator class (`consensus.py:58-82`):

```
class ConsensusAlgorithm:
    """Execute the consensus algorithm marking blocks and transactions as either executed or void

    The consensus algorithm uses the metadata voided_by to set whether a block or transaction is
    If voided_by is empty, then the block or transaction is executed. Otherwise, it is voided.
    """
```

So: **`voided_by` empty (or `None`) → executed. `voided_by` non-empty → voided.** That is the whole state model.

> **Recap — voiding is marking, not deleting (full treatment in Ch. 10 §10.3).** When a vertex loses a conflict, the node does **not** erase it. It stays in storage, fully present, with its `voided_by` set populated to record why it lost (which hashes are causing its voidance). The reason is reversibility: consensus decisions are provisional. A transaction that loses today can win tomorrow if a heavier transaction confirming it arrives, and a chain of blocks abandoned now can be re-adopted if it later overtakes the current best chain. You can only "un-void" something you kept. → full treatment in Ch. 10.

Two engines, one interface

There is a second reason this package exists as a unit: Hathor supports **two consensus mechanisms**, and the node must be able to run either one.

- **Proof-of-Work** (the default, public mainnet/testnet): blocks are produced by mining, and the "real" history is the one backed by the most computational work. This is what almost all of `block_consensus.py` and `transaction_consensus.py` implement.
- **Proof-of-Authority** (private networks): there is no mining. A fixed, configured set of signers take turns producing blocks, each block carrying a cryptographic signature instead of a proof-of-work nonce. The `poa/` subpackage implements this.

Which engine a node uses is a settings choice, `CONSENSUS_ALGORITHM`, defaulting to `PowSettings()` (`conf/settings.py:65`). We cover PoW first, in depth, then PoA as a focused variant in §32.7.

32.3 The concepts it rests on

Before walking the code, three ideas from earlier chapters need to be fresh. Each gets a recap box, not a re-teaching.

> **Recap — accumulated weight and the heaviest-DAG rule (full treatment in Ch. 9).** Every vertex carries a **weight**[3]: a number measuring how much proof-of-work it represents (weight = $\log_2$ of the expected number of hash attempts). The **accumulated weight** of a vertex is the total work of the vertex plus all the work that has piled up behind it — every later vertex that confirms it adds its own weight to the pile. Hathor's consensus rule is therefore the heaviest rule, not the longest rule: when two histories compete, the one with more accumulated work wins, because reproducing that work is what an attacker would have to redo to rewrite it. The metadata fields are `accumulated_weight` (`transaction_metadata.py:48`) and, for blocks, `score` (`:49`). → full treatment in Ch. 9.

> **Recap — the vertex graph and its two edge types (full treatment in Ch. 8 & Ch. 25).** A vertex is any node of Hathor's ledger graph — a `Block` or a `Transaction`. Vertices are linked by **two distinct kinds of edge**: parents (the confirmation/verification DAG — "I confirm these vertices") and inputs (the funds DAG — "I spend this output"). Consensus walks **both** graphs: voiding has to flow to everything that confirms a voided vertex and everything that spends a voided vertex's outputs. → full treatment in Ch. 8 (the model) and Ch. 25 (the code: `parents`, `TxInput`, `TxOutput`).

> **Recap — graph traversal: BFS over the DAG (full treatment in Ch. 8 & Ch. 27).** To propagate a decision to "everything downstream," consensus repeatedly performs a **breadth-first walk**[4] over the DAG, starting from a vertex and visiting its descendants in timestamp order. The codebase's tool for this is `BFSTimestampWalk` (`hathor/transaction/storage/traversal.py`), which can be told to follow the funds DAG, the verification DAG, or both, and to walk left-to-right (toward descendants) or right-to-left (toward ancestors). You will see it used in almost every propagation method below. → full treatment of DAG traversal in Ch. 27.

With those in hand, the code becomes readable.

32.4 The code, walked: the orchestrator

The public entry point is one method: `ConsensusAlgorithm.unsafe_update` (`consensus.py:132`). The vertex handler calls it exactly once per accepted vertex.

```python
def unsafe_update(self, base: BaseTransaction) -> list[ConsensusEvent]:
    assert self.tx_storage.is_only_valid_allowed()
    meta = base.get_metadata()
    assert meta.validation.is_valid()          # consensus assumes verification already passed
    ...
    context = self.create_context()
    ...
    if isinstance(base, Transaction):
        context.transaction_algorithm.update_consensus(base)
    elif isinstance(base, Block):
        context.block_algorithm.update_consensus(base)
```

Three things to read off this:

1. **The "unsafe" in the name is a contract, not a warning about bugs.** The docstring (`consensus.py:133-138`) says the caller is responsible for crashing the full node if this method throws. Consensus mutates a lot of metadata across many vertices; a half-applied update would leave the ledger inconsistent. So if anything goes wrong mid-update, the node must not limp on — it must crash and restart from the last consistent on-disk state. The vertex handler honors this: on any exception it marks the vertex with `CONSENSUS_FAIL_ID` and calls `crash_and_exit` (`vertex_handler.py:178-183`).
2. **The block/transaction split is dispatched right here.** A `Block` and a `Transaction` need genuinely different consensus logic — blocks compete as chains ranked by score; transactions compete as conflicts ranked by accumulated weight. So there are two algorithm objects, `block_algorithm` and `transaction_algorithm`, and `unsafe_update` routes to the right one by type. Everything in §32.5 is the transaction side; §32.6 is the block side.
3. **A fresh context object is created per call.** `create_context()` (`consensus.py:127`) returns a `ConsensusAlgorithmContext` that lives only for the duration of this one update.

The context object

`ConsensusAlgorithmContext` (`context.py:42`) is the scratchpad for a single consensus run. It holds the two algorithm instances, the running set of affected transactions, and the optional reorg record:

```python
class ConsensusAlgorithmContext:
    consensus: 'ConsensusAlgorithm'
    block_algorithm: 'BlockConsensusAlgorithm'
    transaction_algorithm: 'TransactionConsensusAlgorithm'
    txs_affected: set[BaseTransaction]
    reorg_info: ReorgInfo | None
    ...
    def save(self, tx: BaseTransaction) -> None:
        """Only metadata is ever saved in a consensus update."""
        assert tx.storage is not None
        self.txs_affected.add(tx)
        tx.storage.save_transaction(tx, only_metadata=True)
```

Two design points worth pausing on:

- **`save()` only ever writes metadata** (`context.py:72-76`). Consensus never changes a vertex's contents — the inputs, outputs, signatures are immutable facts. It only changes the node's opinion about the vertex (its `voided_by`, `score`, `first_block`). Keeping that invariant in one method makes it impossible to accidentally rewrite history while reasoning about it.
- **`txs_affected` accumulates every vertex touched** so the orchestrator can, after the core logic finishes, update the indexes and publish events for exactly those vertices (`consensus.py:166-167, 222-227`). Consensus does the thinking; the orchestrator does the bookkeeping afterward.

`ReorgInfo` (`context.py:35-39`) is a small frozen record naming the three blocks that define a reorg — the `common_block` where the chains diverged, the `old_best_block`, and the `new_best_block`. It is set at most once per run via `mark_as_reorg` (`context.py:78-81`), which asserts it was not already set.

32.5 Transaction consensus: resolving conflicts

This is the heart of the chapter. The driver is `TransactionConsensusAlgorithm.update_consensus` (`transaction_consensus.py:49`):

```python
def update_consensus(self, tx: Transaction) -> None:
    self.mark_inputs_as_used(tx)
    self.update_voided_info(tx)
    self.set_conflict_twins(tx)
    self.execute_nano_contracts(tx)
```

We will take the first three in order. (`execute_nano_contracts` is a no-op for plain transactions — `:55-61` — because nano-contract execution happens when a block confirms the transaction, not when the transaction itself arrives. That belongs to Ch 39.)

Step 1 — detecting the conflict

When a transaction spends an output, consensus records that the output is now spent by this transaction. If the output was **already** spent by some other transaction, the two transactions are in conflict — a **double-spend**[5].

`mark_input_as_used` (`transaction_consensus.py:69`) does the detection:

```
def mark_input_as_used(self, tx: Transaction, txin: TxInput) -> None:
    spent_tx = tx.storage.get_transaction(txin.tx_id)
    spent_meta = spent_tx.get_metadata()
    spent_by = spent_meta.spent_outputs[txin.index]      # who else spends this output?
    assert tx.hash not in spent_by

    meta = tx.get_metadata()
    if spent_by:                                         # someone already does → conflict!
        # We initially void ourselves. This conflict will be resolved later.
        if not meta.voided_by:
            meta.voided_by = {tx.hash}
        else:
            meta.voided_by.add(tx.hash)
        if meta.conflict_with:
            meta.conflict_with.extend(set(spent_by) - set(meta.conflict_with))
        else:
            meta.conflict_with = spent_by.copy()
    ...
    spent_by.append(tx.hash)                             # record ourselves as a spender
```

Read the key move carefully: **a newly-arrived transaction that conflicts voids itself first** (`meta.voided_by = {tx.hash}`). It does not assume it is the winner. It also records the hashes it conflicts with in `conflict_with` (`transaction_metadata.py:44`), and tells each conflicting transaction about itself in turn (`:93-103`). The actual winner is decided later, in `check_conflicts`. Voiding-yourself-then-competing keeps the algorithm symmetric: whether you arrived first or second, the same comparison decides the outcome.

> **Recap — within-tx vs. cross-tx double spends (full treatment in Ch. 31 §double-spend).** Two outputs spent by the same transaction at once is a structural error — verification rejects it as **invalid**, and it never reaches consensus. Two different transactions spending the same output is a **conflict** — both are valid in isolation, both reach consensus, and this method is where the node notices. Invalid is verification's job; conflict is consensus's job. → full treatment in Ch. 31.

Step 2 — computing voided_by from the surroundings

`update_voided_info` (`transaction_consensus.py:177`) is the longest method on the transaction side. Its job: compute this transaction's `voided_by` from its context, then trigger conflict resolution. It works in layers.

First it **inherits voiding from parents and inputs**. If any vertex this transaction confirms (a parent) or spends from (an input) is voided, this transaction must inherit that voidance — you cannot be part of the canonical ledger if you build on something that is not:

```
voided_by: set[bytes] = set()

# Union of voided_by of parents
for parent in tx.get_parents():
    parent_meta = parent.get_metadata()
    if parent_meta.voided_by:
        voided_by.update(
            self.context.consensus.filter_out_voided_by_entries_from_parents(parent, parent_meta.
        )
...
# Union of voided_by of inputs
for txin in tx.inputs:
    spent_tx = tx.storage.get_transaction(txin.tx_id)
    spent_meta = spent_tx.get_metadata()
    if spent_meta.voided_by:
        voided_by.update(spent_meta.voided_by)
```

This is the **propagation invariant** stated in the orchestrator docstring (`consensus.py:71-73`): the `voided_by` of a vertex is always a subset of the `voided_by` of everything that confirms it or spends from it. Voidance flows strictly downstream.

Then it **adds its own hash if it is losing a conflict.** A transaction's own hash appears in its `voided_by` only when it has a conflict and is not (yet) the winner (`:224-225`):

```
if meta.conflict_with:
    voided_by.add(tx.hash)
```

Finally, having settled its own `voided_by`, it **kicks off conflict resolution** for the transactions involved (`:259-262`):

```
meta = tx.get_metadata()
if meta.voided_by == {tx.hash}:      # voided ONLY by itself → it is a live conflict candidate
    self.check_conflicts(tx)
```

The guard `voided_by == {tx.hash}` is precise: a transaction is a candidate to win a conflict only if the sole reason it is voided is its own conflict — not because a parent or input is also voided. If it is voided for some other reason too, it cannot win regardless, so there is nothing to resolve.

Step 3 — picking the winner: `check_conflicts`

`check_conflicts` (`transaction_consensus.py:302`) is the method Chapter 10 promised. It implements the **heavier-wins** rule:

```python
def check_conflicts(self, tx: Transaction) -> None:
    """Check which transaction is the winner of a conflict, the remaining are voided."""
    meta = tx.get_metadata()
    if meta.voided_by != {tx.hash}:
        return

    # gather conflicting transactions that are still candidates
    candidates: list[Transaction] = []
    conflict_list: list[Transaction] = []
    for h in meta.conflict_with or []:
        conflict_tx = cast(Transaction, tx.storage.get_transaction(h))
        conflict_list.append(conflict_tx)
        conflict_tx_meta = conflict_tx.get_metadata()
        if not conflict_tx_meta.voided_by or conflict_tx_meta.voided_by == {conflict_tx.hash}:
            candidates.append(conflict_tx)
```

It then checks whether `tx` has the highest accumulated weight among the candidates, in two passes — first against already-voided candidates (`:327-335`), then against executed ones (`:337-351`):

```python
# Compare against executed candidates
tie_list = []
for candidate in candidates:
    tx_meta = candidate.get_metadata()
    if not tx_meta.voided_by:
        candidate.update_accumulated_weight(stop_value=meta.accumulated_weight)
        tx_meta = candidate.get_metadata()
        d = tx_meta.accumulated_weight - meta.accumulated_weight
        if d == 0:
            tie_list.append(candidate)
        elif d > 0:
            is_highest = False
            break
if not is_highest:
    return

# We won or tied: void every conflicting tx...
for conflict_tx in sorted(conflict_list, key=lambda x: x.timestamp, reverse=True):
    self.mark_as_voided(conflict_tx)

if not tie_list:
    # ...and if it wasn't a tie, declare ourselves the winner.
    self.mark_as_winner(tx)
```

The decision rule, plainly:

- **Strictly heavier than every rival → win.** `mark_as_winner` removes the transaction's own hash from `voided_by`, un-voiding it (`:362-372`).
- **Strictly lighter than some rival → lose.** Stay voided; do nothing more.
- **Exact tie on accumulated weight → both stay voided.** Note the subtlety: in a tie, the candidate is added to `tie_list`, the rivals are voided, but `mark_as_winner` is not called — so nobody wins. A tie resolves to "all conflicting transactions remain voided" until some later transaction breaks the tie by piling more weight behind one of them. The system refuses to guess.

The invariant the algorithm maintains is asserted in `assert_valid_consensus` (`:291-300`): **two transactions that conflict can never both be executed.** That is the whole point — at most one spender of an output is ever live.

> **On the tie-break by timestamp.** The grounding notes (and Ch 10) mention a timestamp tie-break. Read the code precisely: timestamp is used to order the voiding of the conflict list — `sorted(conflict_list, key=lambda x: x.timestamp, reverse=True)` at `:355` — not to crown a winner. An exact accumulated-weight tie does **not** elect the older transaction; it leaves both voided. Timestamp only determines the order in which the losers are processed. (Block consensus does use a hash tie-break to pick a chain — §32.6 — but transaction consensus deliberately does not pick a winner on a weight tie.)

Step 4 — propagating the void: the BFS

When a transaction loses, its voidance must reach **everything downstream of it** — every transaction that confirms it and every transaction that spends one of its outputs. That is what `add_voided_by` does (`transaction_consensus.py:462`), via a breadth-first walk over both DAGs:

```
def add_voided_by(self, tx: Transaction, voided_hash: bytes, *, is_dag_verifications: bool = True
    ...
    bfs = BFSTimestampWalk(tx.storage, is_dag_funds=True, is_dag_verifications=is_dag_verificatio
                           is_left_to_right=True)
    check_list: list[Transaction] = []
    for tx2 in bfs.run(tx, skip_root=False):
        meta2 = tx2.get_metadata()
        if tx2.is_block:
            assert isinstance(tx2, Block)
            self.context.block_algorithm.mark_as_voided(tx2)
        ...
        if meta2.voided_by:
            meta2.voided_by.add(voided_hash)
        else:
            meta2.voided_by = {voided_hash}
        ...
        self.context.save(tx2)
        bfs.add_neighbors()
```

The walk starts at the loser and visits every descendant in timestamp order (`is_left_to_right=True` — toward newer vertices), stamping `voided_hash` into each one's `voided_by`. The mirror-image method `remove_voided_by` (`:374`) does the reverse walk when a transaction is un-voided (e.g. because it just won a conflict), peeling the hash back out of every descendant and re-checking their conflicts in case un-voiding revives a downstream winner.

This BFS is the mechanism behind the docstring's promise: if tx1 is voided and tx2 verifies tx1, then tx2 must be voided as well (`consensus.py:67-69`).

A hand-traced example

Take a concrete double-spend. Alice has one unspent output, `U`, worth 10 HTR.

```
                ┌──────────────────────────── tx_A  (weight 20, spends U)
U ──────────────┤
                └──────────────────────────── tx_B  (weight 22, spends U)

tx_C  spends an output of tx_B
```

Two transactions, `tx_A` and `tx_B`, both spend `U`. Both are individually valid (Alice signed both). `tx_C` later spends an output of `tx_B`.

Trace what consensus does, assuming `tx_A` arrives first, then `tx_B`, then `tx_C`:

1. **`tx_A` arrives.** `mark_input_as_used` sees `U` has no spender yet, so no conflict. `tx_A` is executed (`voided_by` stays empty). `U` now records `spent_by = [tx_A]`.
2. **`tx_B` arrives.** `mark_input_as_used` sees `U.spent_by = [tx_A]` — conflict. `tx_B` voids itself (`voided_by = {tx_B}`) and records `conflict_with = [tx_A]`; `tx_A` learns `conflict_with = [tx_B]` too. Then `update_voided_info` calls `check_conflicts(tx_B)`. `tx_B`'s accumulated weight (22) is compared against the executed candidate `tx_A` (20). `tx_B` is strictly heavier, so: `mark_as_voided(tx_A)` voids `tx_A`, and `mark_as_winner(tx_B)` un-voids `tx_B`. **The ledger flips: `tx_B` executed, `tx_A` voided.**
3. **`tx_C` arrives** spending `tx_B`'s output. Since `tx_B` is executed (not voided), `tx_C` inherits no voidance and is itself executed.

Now imagine the reverse order — `tx_B` first, then `tx_A`. `tx_B` executes. `tx_A` arrives, conflicts, voids itself, and `check_conflicts(tx_A)` finds `tx_A` (20) is lighter than the executed `tx_B` (22). `tx_A` stays voided; nothing flips. **Same final state, regardless of arrival order** — which is exactly the determinism the network needs. Every node, however it received the two transactions, ends with `tx_B` and `tx_C` executed and `tx_A` voided.

And had `tx_A` already been confirmed and `tx_C`-like descendants built on it before `tx_B` arrived? The `add_voided_by` BFS would walk from `tx_A` forward, voiding every descendant that hangs off it. That cascade is why voiding has to be a graph walk and not a single flag flip.

32.6 Block consensus: chains, score, and reorgs

Blocks play a different game. Where transactions compete pairwise over a shared input, blocks compete as **chains** — and the winning chain is the one with the highest **score**.

Score: accumulated work of a whole sub-DAG

The block-side weight metric is `score`, computed by `calculate_score` (`block_consensus.py:577`) and its recursive helper `_score_block_dfs` (`:512`). The docstring (`:120-123`) defines it:

> The score of a block is calculated as the sum of the weights of all transactions and blocks both directly and indirectly verified by the block.

So a block's score is the total work of everything behind it in the DAG — every ancestor block, and every transaction those blocks confirm. The score of a chain is the score of its head block. Crucially, score is **monotonic**: each new block on top adds its own work plus the work of the transactions it newly confirms, so a longer-standing, more-confirmed chain naturally accrues a higher score. The code even relies on score being **immutable once computed** for a given block — the sub-DAG behind a block never changes, so the score can be cached and only re-checked for consistency (`:562-573`).

`score` is stored in metadata (`transaction_metadata.py:49`), distinct from a transaction's `accumulated_weight`.

Selecting the best chain

`BlockConsensusAlgorithm.update_voided_info` (`block_consensus.py:109`) is the block-side counterpart to the transaction method of the same name. Its enormous docstring (`:109-159`) enumerates eight cases for how an arriving block can attach to the existing block tree — to the head or tail of the best chain or a side chain, with single or multiple best chains. The common ones collapse to a simple comparison.

When the new block attaches to the head of the current best chain (the usual case), it extends the winner and there is nothing to decide (`:196-203`). When it attaches anywhere else — building on a side chain[6] — the node must ask whether that side chain has now overtaken the best chain:

```
# First, void this block (otherwise it would always look like a head).
self.mark_as_voided(block, skip_remove_first_block_markers=True)

head = storage.get_best_block()
head_meta = head.get_metadata(force_reload=True)
best_score = head_meta.score

score = self.calculate_score(block)

winner = False
if score > best_score:
    winner = True
elif score == best_score:
    # Use block hashes as a tie breaker.
    if block.hash < head.hash:
        winner = True
```

Read off the **score comparison** (`block_consensus.py:227-232`):

- **New chain's score strictly greater than the current best → the new chain wins.**
- **Exact tie → break by block hash** (the lexicographically smaller hash wins). Unlike transaction conflicts, block ties are broken deterministically, because the chain has to make progress — leaving two competing heads live indefinitely would stall block production. The hash is an arbitrary-but-deterministic tiebreak that every node computes identically.
- **Less → the new block stays voided** on its side chain.

When the new block wins, the algorithm voids the old head's chain down to the fork point and promotes the new chain (`:244-263`):

```
# Winner, winner, chicken dinner!
common_block = self._find_first_parent_in_best_chain(block)
self.add_voided_by_to_multiple_chains([head], common_block)
self.update_score_and_mark_as_the_best_chain_if_possible(block)
...
if common_block != head:
    self.mark_as_reorg_if_needed(common_block, block)
storage.indexes.height.update_new_chain(height, block)
```

Reorgs

A **reorg** is what just happened in that last block: the node switched its best chain from one branch to another, abandoning blocks it previously considered canonical. `mark_as_reorg_if_needed` (`block_consensus.py:265`) records it:

```python
def mark_as_reorg_if_needed(self, common_block: Block, new_best_block: Block) -> None:
    """Mark as reorg only if reorg size > 0."""
    _, old_best_block_hash = storage.indexes.height.get_height_tip()
    old_best_block = storage.get_transaction(old_best_block_hash)
    reorg_size = old_best_block.get_height() - common_block.get_height()
    if reorg_size == 0:
        assert old_best_block.hash == common_block.hash
        return
    self.context.mark_as_reorg(ReorgInfo(
        common_block=common_block,
        old_best_block=old_best_block,
        new_best_block=new_best_block,
    ))
```

The **reorg size** is how many blocks deep the old chain is abandoned — the height difference between the old best block and the common ancestor (`common_block`) where the two chains diverged. A reorg of size 0 (the chains share their tip) is not a reorg at all and is skipped. When a real reorg is recorded, the orchestrator later emits `REORG_STARTED` and `REORG_FINISHED` events (`consensus.py:210-220, 250-251`) so downstream systems can react, and it re-checks the mempool: transactions that were valid on the old chain but became invalid on the new one (for example, a reward that is no longer unlocked at the new height) are found and removed by `_compute_vertices_that_became_invalid` (`consensus.py:357`).

> **Recap — finality is probabilistic (full treatment in Ch. 10 §finality).** Because a heavier competing chain can always, in principle, arrive and trigger a reorg, no block is ever absolutely final — only increasingly improbable to be reversed as more work piles on top of it. This is why wallets wait for a number of confirmations before treating a payment as settled. The hard floor on reorgs is the **checkpoint**[7] mechanism (verified during verification, Ch 31): the node refuses any reorg that would rewrite history before a hard-coded checkpoint height. → full treatment in Ch. 10.

The asymmetry: why block voiding differs from transaction voiding

This is the single most misunderstood part of the package, and the orchestrator docstring calls it out explicitly (`consensus.py:75-81`):

```
# When a block is not in the bestchain, its voided_by contains its hash. This hash is also propag
# through the transactions that spend one of its outputs.
#
# Differently from transactions, the hash of the blocks are not propagated through the voided_by
# other blocks. For example, if b0 <- b1 <- b2 <- b3 is a side chain, i.e., not the best blockcha
# then b0's voided_by contains b0's hash, b1's voided_by contains b1's hash, and so on. The hash
# b0 will not be propagated to the voided_by of b1, b2, and b3.
```

Spell out the contrast:

- **Transaction voiding propagates the loser's hash downstream.** If `tx1` loses, every descendant gets `tx1`'s hash stamped into its `voided_by`. A voided transaction's hash spreads through the graph.

- **Block voiding does not propagate a block's hash to later blocks.** Each block on a side chain carries only its own hash in `voided_by`. `b1` does not inherit `b0`'s hash.

Why the difference? Because the two kinds of vertex answer different questions. A transaction's `voided_by` records the full set of reasons it is not canonical, and those reasons must travel downstream so any descendant can see exactly what is wrong. A block on a side chain is voided for one self-contained reason — it is on a side chain — and that reason is fully expressed by the block's own hash being present. Each side-chain block is independently "not on the best chain"; there is no useful information in copying `b0`'s hash into `b1`, because `b1` already says "I am off-chain" via its own hash. What does propagate from a voided block is the voidance of any **transaction** that spends one of its outputs — `add_voided_by` (`block_consensus.py:414`) walks the block's spent outputs and voids the spending transactions:

```
spent_by: Iterable[bytes] = chain(*meta.spent_outputs.values())
for tx_hash in spent_by:
    tx = storage.get_transaction(tx_hash)
    self.context.transaction_algorithm.add_voided_by(tx, voided_hash)
```

So when a chain is abandoned, the blocks themselves are quietly marked off-chain, but any transaction that depended on those blocks' rewards or confirmations gets the full downstream voiding treatment from §32.5. The block tree is a tree; the transaction graph is a graph; their voiding rules differ because their topologies differ.

first_block: where a transaction sits in block-time

One more block-side concept connects the two halves. When a block is marked as the best chain, `_score_block_dfs` sets each newly-confirmed transaction's `first_block` field (`block_consensus.py:554-557`) to that block's hash. `first_block` (`transaction_metadata.py:50`) records which block first confirmed this transaction on the best chain — effectively, the transaction's position in block-ordered time. During a reorg, `remove_first_block_markers` (`block_consensus.py:475`) clears these for the abandoned chain, because those transactions are no longer confirmed by any best-chain block until they are re-confirmed by the new one. This is how a transaction knows whether it is still in the mempool (`first_block is None`) or settled into a block.

32.7 The other engine: Proof-of-Authority

Everything above assumes **Proof-of-Work** — blocks earn their place by burning computation, and the heaviest chain wins. Hathor's `poa/` subpackage replaces that with **Proof-of-Authority**, used on private networks where you do not want (or cannot afford) real mining.

> **Recap — Proof-of-Authority (full treatment: this section is the canonical one).** In PoA there is no mining and no open competition to produce blocks. Instead a **fixed, configured set of signers** — identified by their public keys — are the only parties permitted to produce blocks, and they take turns. A block is "earned" not by proof-of-work but by a valid **digital signature** from an authorized signer. It is the consensus model of a permissioned network: you trust a known set of authorities rather than anonymous hash power.

The configuration lives in `consensus_settings.py`. A network is either PoW or PoA, chosen by a discriminated union (`ConsensusSettings`, `:155-161`) on the `type` field. `PoaSettings` (`:116`) carries the signer list:

```
class PoaSettings(_BaseConsensusSettings):
    type: Literal[ConsensusType.PROOF_OF_AUTHORITY] = ConsensusType.PROOF_OF_AUTHORITY
    # A list of Proof-of-Authority signer public keys that have permission to produce blocks.
    signers: tuple[PoaSignerSettings, ...]
```

Each `PoaSignerSettings` (`:99`) is a public key with an optional `start_height`/`end_height` window, so the authorized set can change over time (a signer can be added or retired at a chosen height).

Turns and weight

The clever part is how PoA reuses the same heaviest-chain machinery without any mining. Blocks still have a **weight**, and the block consensus from §32.6 still picks the highest-score chain — but in PoA the weight is assigned by a turn schedule, not by hashing.

`get_signer_index_distance` (`poa.py:58`) computes how far a signer is from being "in turn" for a given height — the expected signer for height `h` is `h % number_of_active_signers` (`:62`), rotating through the set. `calculate_weight` (`poa.py:69`) then assigns block weight from that distance:

```
BLOCK_WEIGHT_IN_TURN = 2.0
BLOCK_WEIGHT_OUT_OF_TURN = 1.0

def calculate_weight(settings: PoaSettings, block: PoaBlock, signer_index: int) -> float:
    index_distance = get_signer_index_distance(settings=settings, signer_index=signer_index, heig
    return BLOCK_WEIGHT_IN_TURN if index_distance == 0 else BLOCK_WEIGHT_OUT_OF_TURN / index_dist
```

The signer whose turn it is produces a weight-2.0 block; an out-of-turn signer produces a lighter block (weight `1.0 / distance`). Because the in-turn block is heavier, the chain built from in-turn blocks accumulates more score, and the existing heaviest-chain rule naturally prefers it. Out-of-turn production is allowed (so the chain does not stall if the scheduled signer is offline) but is "punished" by lower weight, so it only wins if the in-turn signer is absent. PoA gets its ordering for free from the PoW consensus code — it just feeds it a different weight.

Signatures instead of nonces

A PoA block (`PoaBlock`, a `Block` subclass — see Ch 25) carries a signer ID and a signature instead of a proof-of-work nonce. `PoaSigner.sign_block` (`poa_signer.py:99`) signs the block's data with the signer's private key:

```
def sign_block(self, block: PoaBlock) -> None:
    hashed_poa_data = poa.get_hashed_poa_data(block)
    signature = self._private_key.sign(hashed_poa_data, ec.ECDSA(hashes.SHA256()))
    block.signer_id = self._signer_id
    block.signature = signature
```

`verify_poa_signature` (`poa.py:86`) checks, at verification time (Ch 31), that the signature came from a currently-active signer. The `signer_id` is the first two bytes of the hash of the signer's public key (`poa_signer.py:110-114`); it is a non-unique hint that lets the verifier skip most candidates before doing the expensive signature check (`poa_signer.py:82-86`).

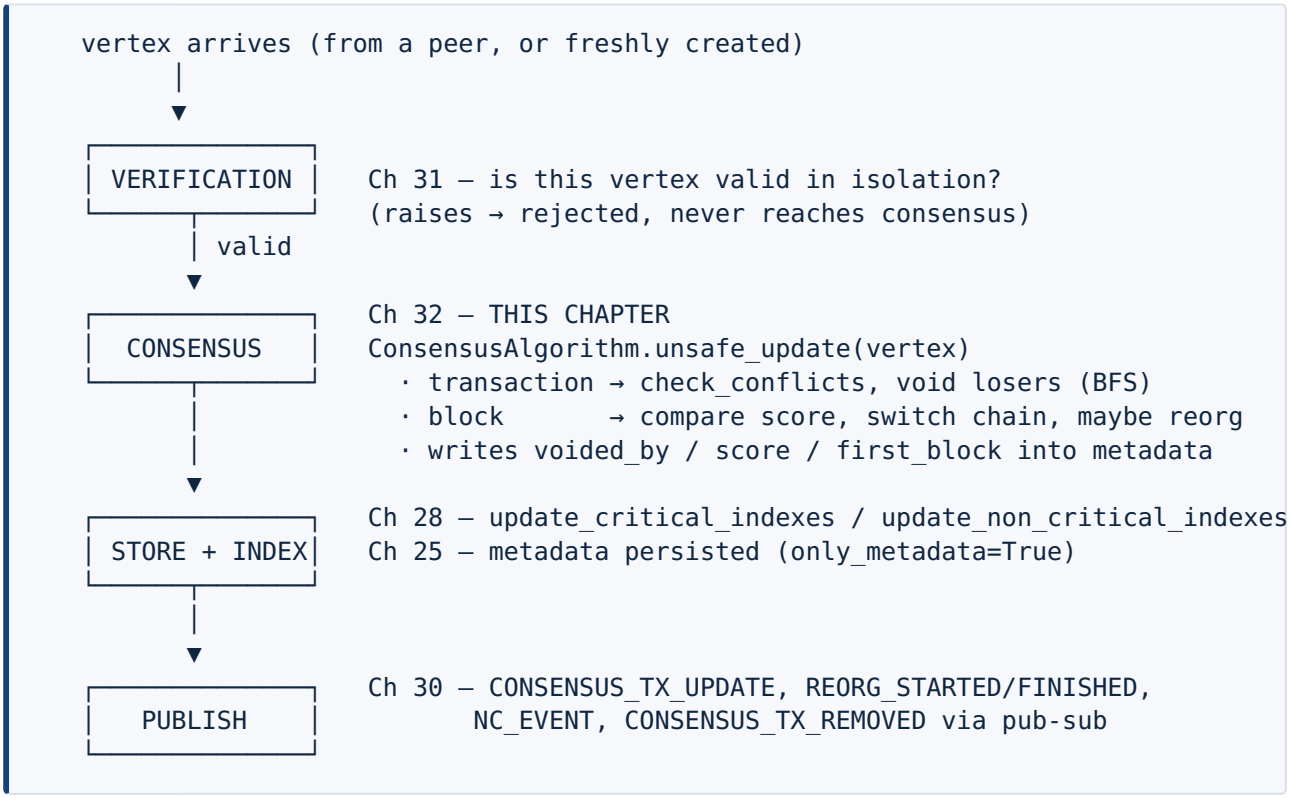
Producing blocks

The mining-equivalent is `PoaBlockProducer` (`poa_block_producer.py:43`) — the PoA node's block factory. It watches for new best blocks, works out whether it is this node's turn (`_get_signer_index`, `:95`), schedules a block at the expected timestamp (`_schedule_block`, `:137`), and produces and signs it (`_produce_block`, `:172`), setting the weight via `poa.calculate_weight` (`:184`). It is the structural analogue of the CPU miner in the PoW world (Ch 37), minus the hashing — there is no puzzle to solve, only a turn to wait for and a signature to apply.

> **Scope note.** This section covers the consensus role of PoA — how it slots into the heaviest-chain machinery and how blocks are authorized. The deeper mechanics of block production timing, signer rotation edge cases, and how PoA interacts with the P2P layer's peer-hello hash (`consensus_settings.py:147-152`) are touched only lightly here; the production side connects forward to mining (Ch 37) and the settings side back to Ch 22.

32.8 How it plugs into the lifecycle

Consensus is not a thing the node runs on a timer. It runs exactly once per accepted vertex, sandwiched into the ingestion pipeline:



Concretely, the **vertex handler** (Ch 33) calls `self._consensus.unsafe_update(vertex)` at `vertex_handler.py:232`, immediately after a successful `validate_full` (`:213`). The handler treats consensus as fallible-and-fatal: any exception from `unsafe_update` triggers `crash_and_exit` (`:178-183`), upholding the "unsafe" contract from §32.4.

The outputs of a consensus run land in three places, all driven by the orchestrator's tail (`consensus.py:222-253`):

- **Metadata** (Ch 25): the `voided_by`, `score`, `accumulated_weight`, and `first_block` fields are persisted via `context.save(...)` (metadata only).
- **Indexes** (Ch 28): for every affected transaction, `update_critical_indexes` and `update_non_critical_indexes` are called so lookups (UTXO, addresses, mempool tips, height) reflect the new canonical view — voided vertices are pulled out of the indexes, un-voided ones put back.
- **Events** (Ch 30): a `list[ConsensusEvent]` is returned and later published through pub-sub — `CONSENSUS_TX_UPDATE` per affected tx, plus `REORG_STARTED`/`REORG_FINISHED`, `CONSENSUS_TX_REMOVED`, and nano-contract execution events when a block confirms contracts.

That is the full circuit: verification gates, consensus decides, storage and indexes record, pub-sub announces.

Recap

Question	Mechanism	Key code
Is a vertex part of the ledger?	`voided_by` empty → executed; non-empty → voided	`transaction_metadata.py:45`
Who decides?	`ConsensusAlgorithm.unsafe_update`, dispatched block vs tx	`consensus.py:132,157-160`
Which conflicting transaction wins?	Highest `accumulated_weight`; exact tie → both stay voided	`transaction_consensus.py:302,327-360`
How does a loss spread?	BFS over funds + verification DAGs, stamping `voided_by`	`transaction_consensus.py:462`
Which chain of blocks is canonical?	Highest `score`; tie → smaller block hash	`block_consensus.py:227-232`
What is a reorg?	Best chain switches; size = height drop to common ancestor	`block_consensus.py:265-282`
Why don't block hashes propagate to later blocks?	Side-chain blocks each carry only their own hash	`consensus.py:75-81`
What is the alternative engine?	Proof-of-Authority: signer turns set block weight	`poa/poa.py:58-72`, `consensus_settings.py:116`
When does consensus run?	Once per vertex, after verification, by the vertex handler	`vertex_handler.py:232`

Consensus is the node's answer to the one question verification cannot answer: given two valid histories, which one is real? Hathor answers it with weight — the heaviest transaction wins a conflict, the heaviest chain wins the blockchain — and records the answer as a reversible mark (`voided_by`) rather than a deletion, so the answer can change as more weight arrives. The block side and the transaction side share a metric (work) but differ in topology and in how voidance spreads, and the whole apparatus can be swapped for Proof-of-Authority on a private network by feeding the same heaviest-chain machinery a turn-based weight instead of a mined one.

The next chapter, **Ch 33 (the vertex handler)**, is the thin but pivotal pipeline that calls verification (Ch 31) and then consensus (Ch 32) in sequence and then commits the result — the place where "a vertex arrived" finally becomes "the ledger changed."

---

1. **Reorg** (reorganization) — when a node abandons part of its current best chain of blocks in favor of a competing chain that has accumulated more work (a higher score). Blocks on the abandoned branch become voided; their transactions return to the mempool or are re-confirmed by the new chain. The reorg size is how many blocks deep the switch goes. ↩
2. **Proof-of-Authority** (PoA) — a consensus model for permissioned networks in which a fixed, configured set of authorized signers (identified by public key) take turns producing blocks, each block carrying a valid signature instead of a proof-of-work nonce. No mining; trust is placed in a known set of authorities. ↩
3. **Weight** — a number measuring how much proof-of-work a vertex represents (weight = $\log_2$ of the expected number of hash attempts). **Accumulated weight** sums a vertex's weight with all the work piled up behind it in the DAG. Consensus prefers the history with the most accumulated weight. Full treatment in Ch. 9. ↩
4. **BFS** (breadth-first search) — a graph-traversal strategy that visits a starting node, then all of its immediate neighbors, then their neighbors, and so on, in expanding "rings." Hathor's `BFSTimestampWalk` visits descendants in timestamp order and is the engine behind voiding propagation. ↩
5. **Double-spend** — two different transactions that both try to spend the same output. Each is valid alone; together they conflict, and consensus must void at least one so a coin is never spent twice. The visible "collision" is exactly what the spent-once rule and `voided_by` machinery exist to catch. ↩
6. **Side chain** — a branch of blocks whose score is lower than the current best chain's. Its head (and the blocks below it that are off the best chain) are voided. A side chain becomes the best chain — triggering a reorg — only if it later overtakes the current best chain's score. ↩
7. **Checkpoint** — a hard-coded `(height, hash)` pair in the node's settings marking a block the network agrees is final. The node refuses any reorg that would rewrite history before a checkpoint, putting a hard floor under otherwise-probabilistic finality. Full treatment in Ch. 10. ↩